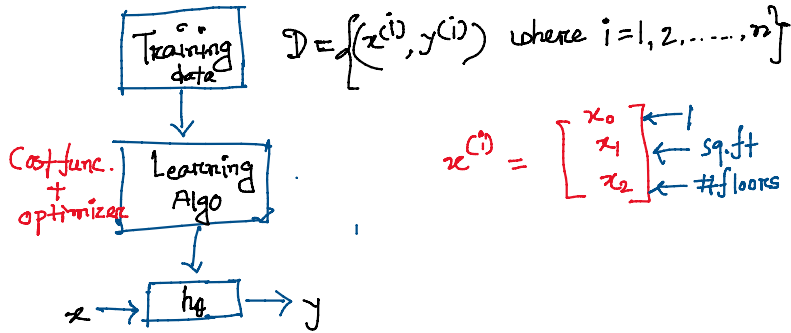


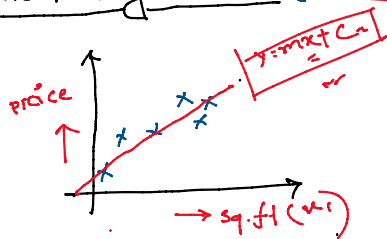
* Supervised learning process:



* Notations:

- Input/feature: $x^{(i)}$
- Output/target: $y^{(i)}$
- Training example: $(x^{(i)}, y^{(i)})$
- Training set: $\{(x^{(i)}, y^{(i)}) \text{ for } i=1, 2, \dots, n\}$
- Test set: $\{x^{*1}, x^{*2}, \dots, x^{*n}\}$
- Hypothesis/model: $h_{\theta}(x)$
- Parameters: $\theta = [\theta_0, \theta_1, \theta_2, \dots, \theta_n]$
- Learning algorithm:
 - cost function
 - optimization algorithm

* Linear regression: (1D data)



The simplest model to fit here is a straight line:

$$y = h_{\theta}(x) = \theta_0 + \theta_1 x$$

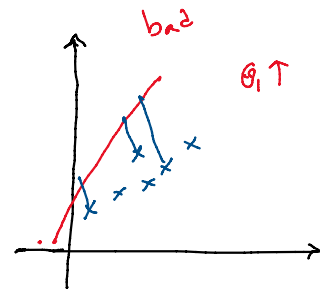
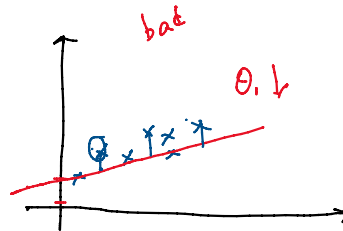
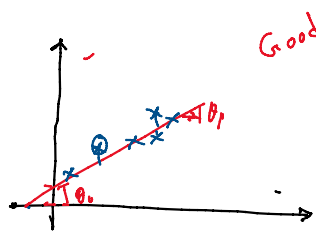
$$\theta^T x, y = \theta_0 x_0 + \theta_1 x_1 = \begin{bmatrix} \theta_0 & \theta_1 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \end{bmatrix}$$

$$\theta^T x, y = \theta^T x \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix}, x = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix}$$

Task: Given $x \rightarrow$ predict y

III. Finding the better approximation of θ :

$h_{\theta}(x^{(i)})$ should be as close to $y^{(i)}$ as possible!

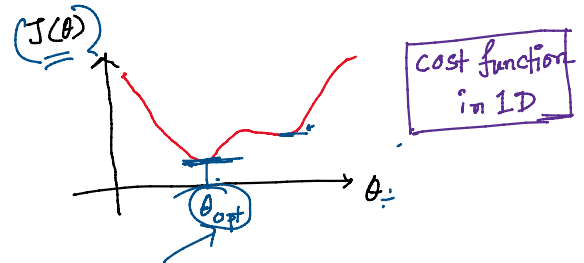


error $J_i(\theta) = (y^{(i)} - h_{\theta}(x^{(i)}))^2 \rightarrow \text{MSE}$

cost function, $J(\theta) = \frac{1}{n} \sum_{i=1}^n J_i(\theta) = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - h_{\theta}(x^{(i)}))^2$

So, we need to find θ that minimizes $J(\theta)$.

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix} \quad \theta_0 \text{ fixed}$$

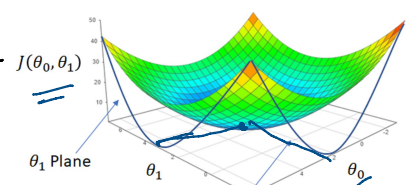


$$\theta_{\text{opt}} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

Optimum model,

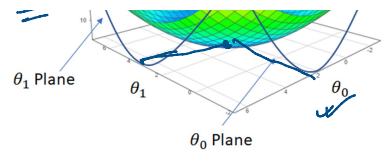
$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

or, $h_{\theta}(x) = 2 + 3x$

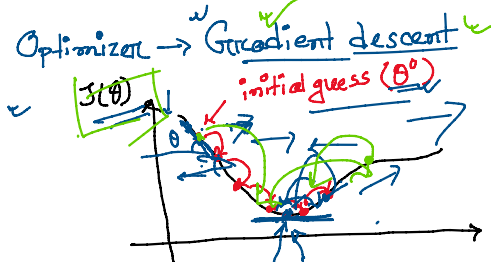


Cost function in 2D

$$\text{or, } h_{\theta}(x) = 2 + 3x$$



Q. How to find θ_{opt} ? [Optimization]



at (i+1)th step,

$$\theta^i = \theta^0 - \frac{\partial J}{\partial \theta}$$

$$\theta^{i+1} = \theta^i - \alpha \frac{\partial J}{\partial \theta}$$

or, $\theta^{i+1} = \theta^i - \alpha \nabla_{\theta} J$ (Stepsize)

for epoch in ():

$$\theta = \theta - \alpha \frac{\partial J}{\partial \theta}$$

10^{-2}
 10^{-3} 10^{-4}

Other cost functions: MSE, MAE, MBE
Other optimizers: Adam, Adaboost, ---

* Linear regression: [Multidimensional data]

x_0	Living area (feet ²)	#bedrooms	Price (1000\$)
	2104	3	400
	1600	3	330
	2400	3	369
	1416	2	232
	3000	4	540
	...	...	...

$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}_{d+1}$$

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_d \end{bmatrix}_{d+1}$$

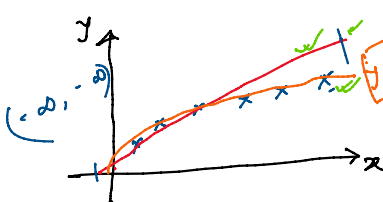
$$y = h_{\theta}(x) = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_d x_d$$

$$= \sum_{i=0}^d \theta_i x_i = \theta^T x$$

cost function, $J(\theta) = \frac{1}{n} \sum_{i=0}^n [y^{(i)} - \theta^T x^{(i)}]^2$ [MSE]

gradient descent, $\theta = \arg \min J(\theta)$
or, $\theta^{i+1} = \theta^i - \alpha \frac{\partial J}{\partial \theta} \nabla_{\theta}(J)$

* Kernel technique & feature engineering:



$$h_{\theta} = \theta^T \phi(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots$$

$$\phi(x) = \begin{bmatrix} x_1 \\ x_2 \\ x_1 x_2 \\ x_1^2 x_2 \end{bmatrix}$$

$$y = h_{\theta} = \theta^T \phi(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 (x_1 x_2) + \theta_4 x_1^2$$

Let's say, we are given length & width of houses respectively as x_1, x_2
... all depend rather on the area ($x_1 x_2$) than

Let's say, we are given length & width of houses respectively x_1, x_2 .
 Housing price will depend rather on the area (x_1, x_2) than
 the individual dimensions!

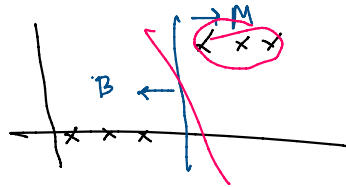
$$x_1, x_2, y = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

$$(x_1, x_2) \rightarrow y = \theta_0 + \theta_1 (x_1 x_2)$$

* Classification:

$$y \in \{0, 1\}$$

logistic regression



$$(-\infty, \infty) \rightarrow (0, 1) \quad g(z) = \frac{1}{1 + e^{-z}}$$

